

End-to-End MLOps Pipeline for Heart Disease Prediction

Submitted By:

2025cs05002 – Kavya Gullipalli

Course:

MLOps (S2-25_AMLCSZG523)

University:

BITS Pilani WILP

Date:

08 May 2026

GitHub Repository:

<https://github.com/2025cs05002Kavya/heart-disease-mlops>

Video Demonstration Link:

<https://drive.google.com/your-link>

Abstract

This project presents an end-to-end MLOps pipeline for heart disease prediction using machine learning and modern DevOps practices. The pipeline includes data preprocessing, exploratory data analysis, model training, experiment tracking using MLflow, FastAPI model deployment, Docker containerization, Kubernetes deployment, CI/CD automation using GitHub Actions, and monitoring/logging implementation. The Random Forest Classifier achieved the best performance and was selected for deployment. The project demonstrates reproducibility, scalability, automation, and production-readiness of machine learning systems.

Table of Contents

1. Introduction	5
2. Dataset and Exploratory Data Analysis	6
3. Feature Engineering and Model Development	8
4. Experiment Tracking using MLflow	10
5. API Development and Model Serving.....	11
6. Docker Containerization.....	13
7. CI/CD Pipeline using GitHub Actions.....	15
8. Kubernetes Deployment and Monitoring	17
9. Conclusion.....	19
References:	20

List of Figures

Figure 1: End-to-End MLOps Architecture Pipeline	5
Figure 2: Correlation Heatmap	6
Figure 3: Class Distribution Plot	7
Figure 4: Feature Histograms.....	7
Figure 5: Logistic Regression Evaluation Metrics	9
Figure 6: Random Forest Evaluation Metrics	9
Figure 7: Confusion Matrix	9
Figure 8: MLflow Experiment Dashboard	10
Figure 9: FastAPI Swagger UI.....	12
Figure 10: Prediction Response from API	12
Figure 11: Docker Build Success	13
Figure 12: Docker Image Verification	14
Figure 13: Docker Container Running	14
Figure 14: GitHub Actions Workflow	15
Figure 15: Successful CI Pipeline Execution	16
Figure 16: Kubernetes Pod Deployment	18
Figure 17: Kubernetes Service Verification	18
Figure 18: API Monitoring and Logging	18

1.Introduction

MLOps combines machine learning, DevOps, and data engineering practices to automate the machine learning lifecycle. It enables reproducibility, scalability, monitoring, testing, and continuous deployment of machine learning systems.

The objective of this project is to build an end-to-end production-ready machine learning pipeline for predicting heart disease using patient health data. The project implements data preprocessing, exploratory data analysis, model training, experiment tracking, API deployment, containerization, CI/CD automation, Kubernetes deployment, and monitoring.

Figure 1: End-to-End MLOps Architecture Pipeline



2.Dataset and Exploratory Data Analysis

The project uses the Heart Disease UCI Dataset obtained from the UCI Machine Learning Repository. The dataset contains medical attributes related to patients such as age, sex, chest pain type, cholesterol level, resting blood pressure, and maximum heart rate achieved.

The target variable indicates the presence or absence of heart disease. The dataset was cleaned and preprocessed by handling missing values and converting features into a suitable format for machine learning models.

Exploratory Data Analysis (EDA) was performed to understand the relationships between features and identify important patterns in the dataset. Histograms were used to visualize feature distributions, correlation heatmaps were used to study feature relationships, and class distribution plots were used to analyze target balance.

Figure 2: Correlation Heatmap

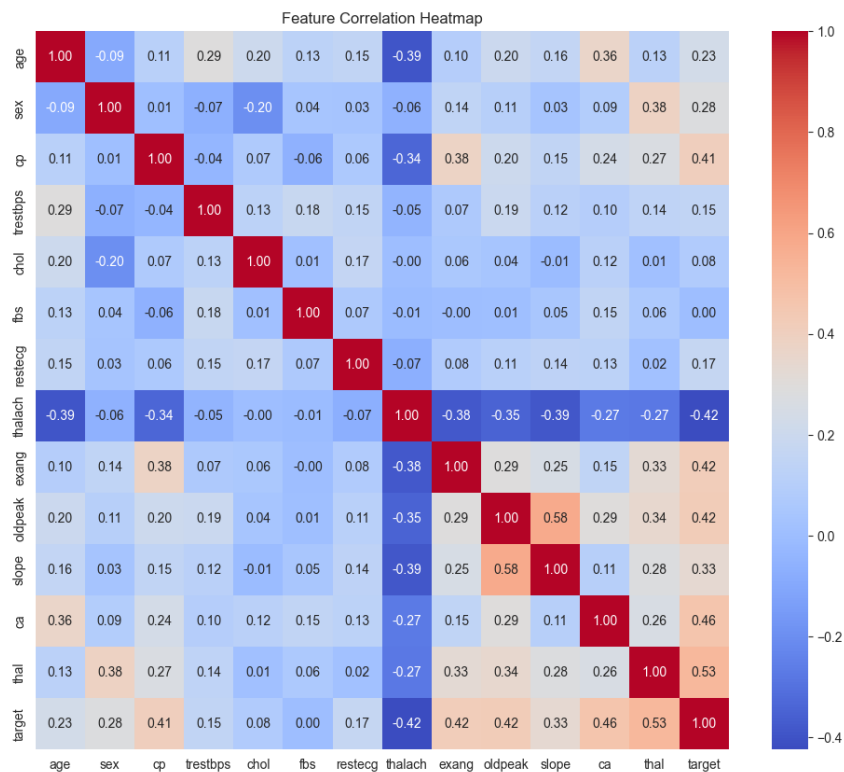


Figure 3: Class Distribution Plot

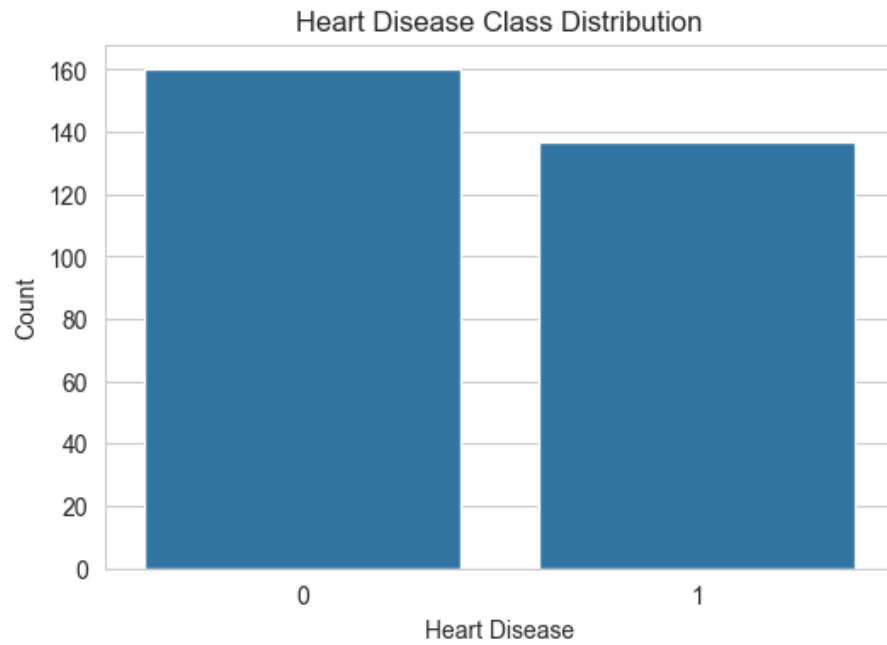


Figure 4: Feature Histograms



3.Feature Engineering and Model Development

Feature engineering and preprocessing were performed before training the machine learning models. The dataset was separated into input features and target labels. Scaling and preprocessing techniques were applied to ensure model compatibility and better performance.

Two machine learning classification algorithms were implemented:

1. Logistic Regression
2. Random Forest Classifier

The models were trained using the processed dataset and evaluated using multiple performance metrics including accuracy, precision, recall, F1-score, and ROC-AUC score.

Random Forest Classifier achieved better overall performance compared to Logistic Regression and was selected as the final model for deployment.

Logistic Regression Results:

- Accuracy: 83.33%
- Precision: 84.61%
- Recall: 78.57%
- F1 Score: 81.48%
- ROC-AUC: 94.97%

Random Forest Results:

- Accuracy: 86.67%
- Precision: 88.46%

- Recall: 82.14%
- F1 Score: 85.18%
- ROC-AUC: 94.53%

Figure 5: Logistic Regression Evaluation Metrics

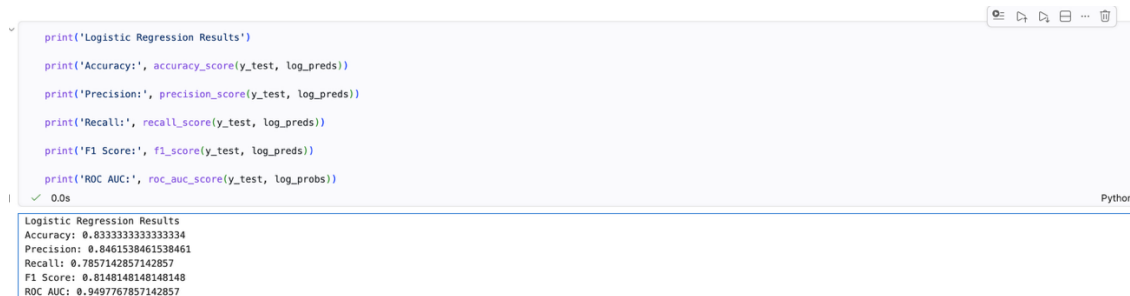


Figure 6: Random Forest Evaluation Metrics

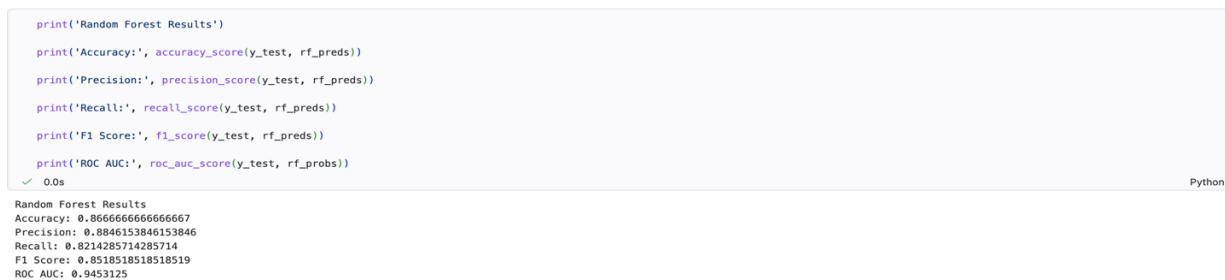
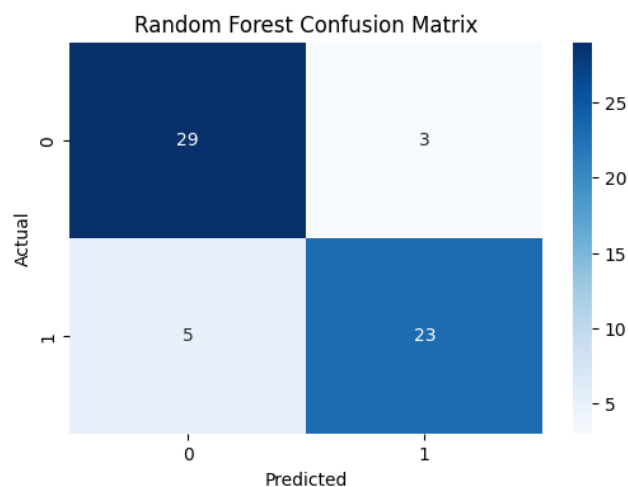


Figure 7: Confusion Matrix



4. Experiment Tracking using MLflow

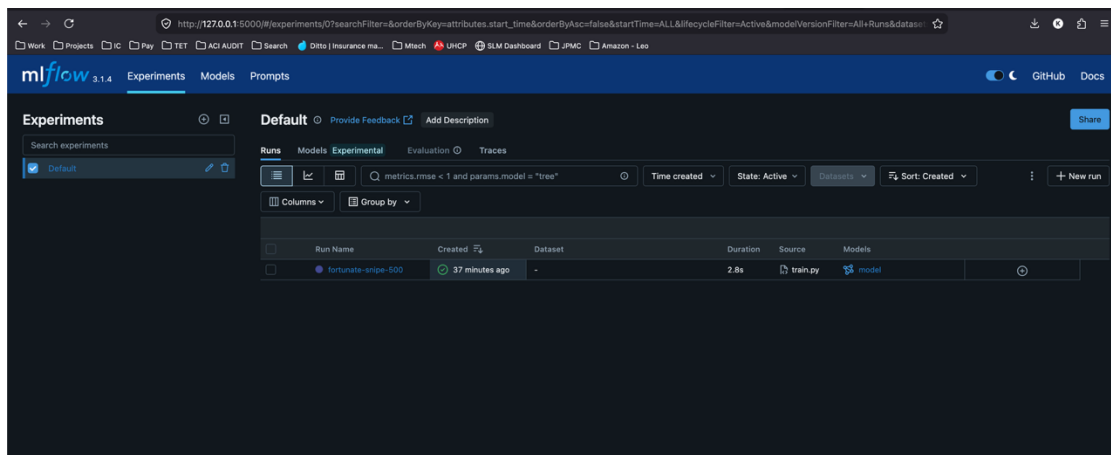
MLflow was integrated into the project for experiment tracking and model management. The training process logged important metrics, parameters, and artifacts for every experiment run.

The following information was tracked using MLflow:

- Model parameters
- Accuracy and evaluation metrics
- Trained model artifacts
- Experiment history

MLflow enabled reproducibility and efficient comparison between different machine learning models and training runs.

Figure 8: MLflow Experiment Dashboard



5.API Development and Model Serving

The trained machine learning model was deployed using FastAPI to provide real-time prediction capabilities through REST APIs.

A `/predict` endpoint was implemented to accept patient health data in JSON format and return:

- Predicted heart disease status
- Prediction confidence score

FastAPI automatically generated interactive API documentation using Swagger UI, which enabled easy testing and validation of API endpoints.

Sample JSON Input:

```
{  
  "age": 63,  
  "sex": 1,  
  "cp": 3,  
  "trestbps": 145,  
  "chol": 233,  
  "fbs": 1,  
  "restecg": 0,  
  "thalach": 150,  
  "exang": 0,  
  "oldpeak": 2.3,  
  "slope": 0,  
  "ca": 0,  
  "thal": 1  
}
```

The API successfully processed input patient data and returned prediction responses with confidence values. This demonstrated successful integration between the machine learning model and the production-serving API.

Figure 9: FastAPI Swagger UI

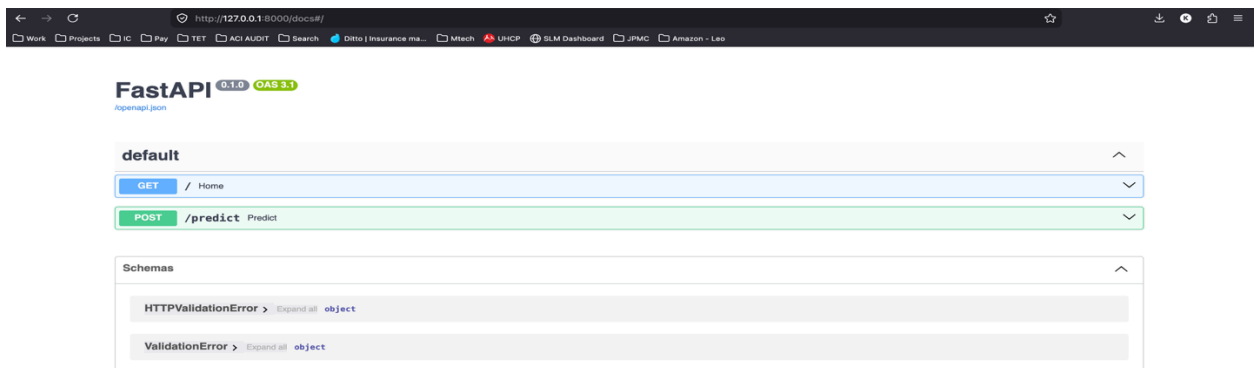


Figure 10: Prediction Response from API



6.Docker Containerization

The FastAPI application and trained machine learning model were containerized using Docker to ensure portability, reproducibility, and isolated execution environments.

A Dockerfile was created to package:

- Python dependencies
- API application
- Trained model
- Project source code

The Docker container exposed port 8000 and successfully served prediction requests inside the containerized environment.

Docker Build Command:

docker build -t heart-api .

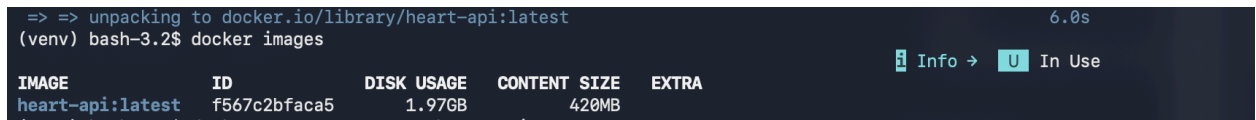
Docker Run Command:

docker run -p 8000:8000 heart-api

Figure 11: Docker Build Success

```
=> [5/5] COPY . . 17.9s => exporting to image
=> => exporting manifest sha256:de8f46b4c3522ae6530c279f98d2e9b5ca4981a03a1ca0b66896c1ffbe9e0a89 0.0s
=> => exporting config sha256:54dfa03fd990a1ba99d96cbb175e40fb5e53f8bae19fa46da27db893253fbbe7 0.0s
=> => exporting attestation manifest sha256:057a1ef9c36128315d6ec513f78b637c25fc318684a970ec2ec1b4 0.0s
=> => exporting manifest list sha256:f567c2bfaca57e673e1e2f8289da17d13fc603c87256ea2f198756cd53ae6 0.0s
=> => naming to docker.io/library/heart-api:latest 0.0s
=> => unpacking to docker.io/library/heart-api:latest 6.0s
```

Figure 12: Docker Image Verification



```
=> => unpacking to docker.io/library/heart-api:latest 6.0s
(venv) bash-3.2$ docker images
```

IMAGE	ID	DISK USAGE	CONTENT SIZE	EXTRA
heart-api:latest	f567c2bfaca5	1.97GB	420MB	

Info → U In Use

Figure 13: Docker Container Running



```
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO: Application startup complete.
INFO: Uvicorn running on http://0.0.0.0:8000 (Press CTRL+C to quit)
INFO: 192.168.65.1:64928 - "GET /docs HTTP/1.1" 200 OK
INFO: 192.168.65.1:64928 - "GET /openapi.json HTTP/1.1" 200 OK
```

7.CI/CD Pipeline using GitHub Actions

A Continuous Integration and Continuous Deployment (CI/CD) pipeline was implemented using GitHub Actions to automate project validation and workflow execution.

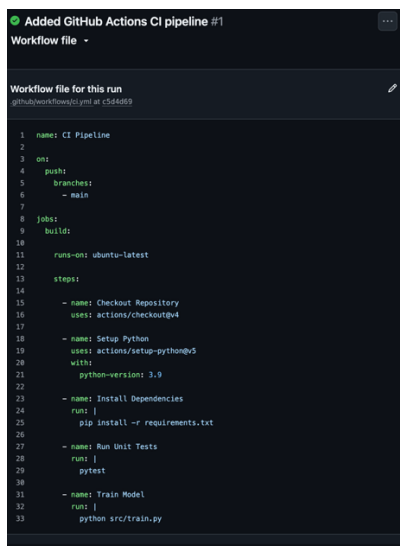
The CI/CD pipeline automatically performs the following tasks whenever code is pushed to the GitHub repository:

- Dependency installation
- Unit testing using Pytest
- Model training execution

This automation ensures reproducibility, early error detection, and streamlined machine learning workflow management.

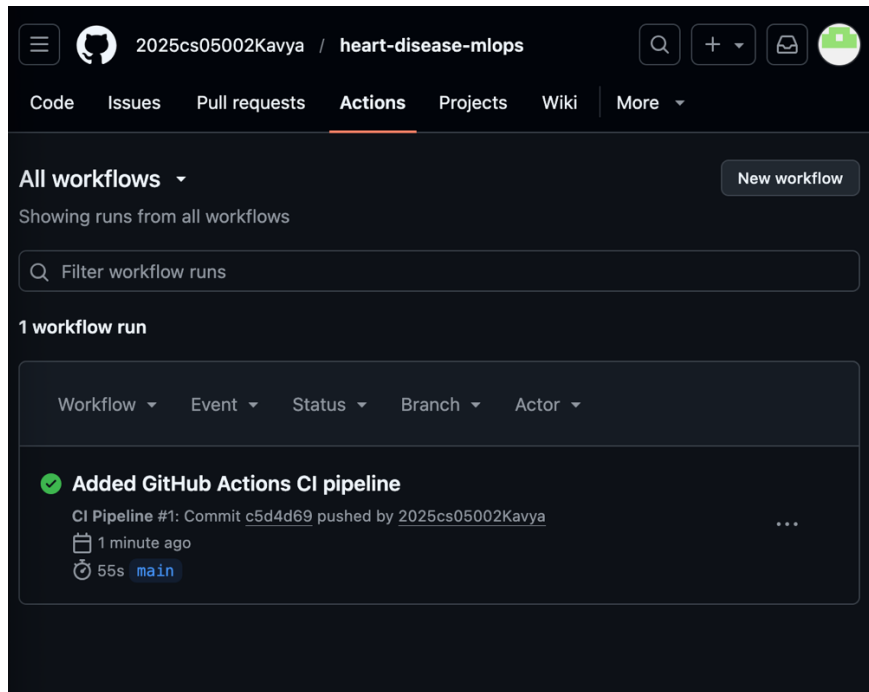
The workflow configuration was defined inside the ``.github/workflows/ci.yml`` file. GitHub Actions successfully executed the workflow and validated the project pipeline with automated testing and training steps.

Figure 14: GitHub Actions Workflow



```
1 name: CI Pipeline
2
3 on:
4   push:
5     branches:
6       - main
7
8 jobs:
9   build:
10
11     runs-on: ubuntu-latest
12
13     steps:
14       - name: Checkout Repository
15         uses: actions/checkout@v4
16
17       - name: Setup Python
18         uses: actions/setup-python@v5
19         with:
20           python-version: 3.9
21
22       - name: Install dependencies
23         run: |
24           pip install -r requirements.txt
25
26       - name: Run Unit Tests
27         run: |
28           pytest
29
30       - name: Train Model
31         run: |
32           python src/train.py
```

Figure 15: Successful CI Pipeline Execution



8.Kubernetes Deployment and Monitoring

The Dockerized FastAPI application was deployed on a local Kubernetes cluster using Docker Desktop Kubernetes.

Kubernetes deployment manifests were created to manage:

- application deployment,
- container orchestration,
- service exposure.

The deployment configuration was defined using `deployment.yaml`, while the service configuration was defined using `service.yaml`.

Kubernetes successfully created and managed the application pod and service for the deployed machine learning API.

Basic monitoring and logging functionality was implemented within the FastAPI application using Python logging.

Prediction requests and confidence scores were logged in the terminal whenever API requests were processed. This enabled simple runtime monitoring and validation of API activity.

Kubernetes Deployment Commands:

```
kubectl apply -f k8s/deployment.yaml
```

```
kubectl apply -f k8s/service.yaml
```

Verification Commands:

```
kubectl get pods
```

```
kubectl get services
```

Figure 16: Kubernetes Pod Deployment

```
(venv) bash-3.2$ kubectl apply -f k8s/deployment.yaml
deployment.apps/heart-api created
(venv) bash-3.2$ kubectl get pods
```

NAME	READY	STATUS	RESTARTS	AGE
heart-api-84496969b9-v4gjq	1/1	Running	0	30s

Figure 17: Kubernetes Service Verification

```
heart-api-8449696b9-v4gjq 1/1 Running 0 30s
(venv) bash-3.2$ kubectl get services
```

NAME	TYPE	CLUSTER-IP	EXTERNAL-IP	PORT(S)	AGE
heart-api-service	LoadBalancer	10.96.90.241	172.18.0.5	80:31439/TCP	9m30s
kubernetes	ClusterIP	10.96.0.1	<none>	443/TCP	11m

Figure 18: API Monitoring and Logging

```
warnings.warn(
/Users/kgullipa/Desktop/heart-disease-mlops/venv/lib/python3.9/site-packages/sklearn/utils/validation.py:2739: UserWarning:
does not have valid feature names, but StandardScaler was fitted with feature names
warnings.warn(
INFO:root:Prediction: 0, Confidence: 0.8130178571428571
INFO: 127.0.0.1:60093 - "POST /predict HTTP/1.1" 200 OK
```

9. Conclusion

This project successfully implemented a complete end-to-end MLOps pipeline for heart disease prediction using modern machine learning and DevOps practices.

The solution included:

- data preprocessing and EDA,
- machine learning model development,
- MLflow experiment tracking,
- FastAPI model serving,
- Docker containerization,
- Kubernetes deployment,
- CI/CD automation,
- monitoring and logging.

The project demonstrated reproducibility, scalability, automation, and production-readiness of machine learning workflows.

Future improvements may include:

- cloud deployment,
- advanced monitoring using Prometheus and Grafana,
- automated retraining pipelines,
- model versioning and drift detection.

References:

1. <https://archive.ics.uci.edu/ml/datasets/heart+Disease>
2. <https://fastapi.tiangolo.com/>
3. <https://mlflow.org/>
4. <https://www.docker.com/>
5. <https://kubernetes.io/>